

Functors

Luca Roversi

Learning outcomes

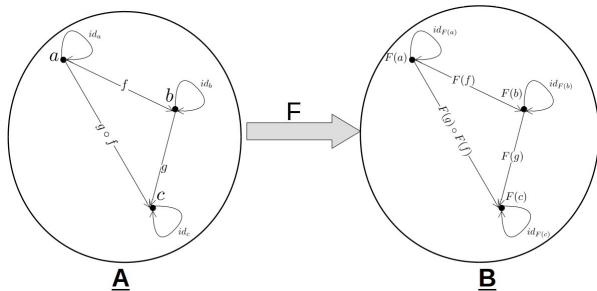
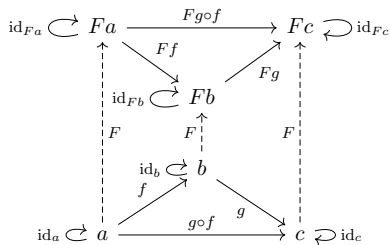
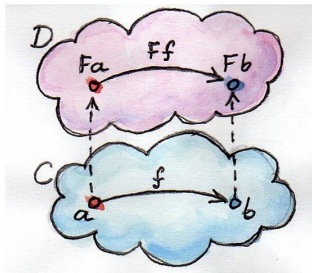
- What is a functor?
- Examples
- Functors in Haskell
- Some exercises

Functor: high level description

- It is a *function* from category to category
- It maps:
 - object to object
 - arrow to arrow

preserving connections

Functors as commuting diagrams



Examples

Identity

$$\begin{array}{ccccc}
 \text{id}_{1a} = \text{id}_a & \xrightarrow{\quad} & 1a = a & \xrightarrow{1f=f} & 1b = b & \xleftarrow{\quad} & \text{id}_{1b} = \text{id}_b & \mathcal{C} \\
 & & \uparrow 1 & & \uparrow 1 & & & \\
 \text{id}_a \hookrightarrow a & & & \xrightarrow{f} & & & b \hookrightarrow \text{id}_b & \mathcal{C}
 \end{array}$$

Constant

$$\begin{array}{ccccc}
 & & Ka = k = Kb & \xleftarrow{\quad} & \text{id}_{Ka} = Kf = \text{id}_{Ka} = \text{id}_k & \mathcal{D} \\
 & \nearrow K & & \nwarrow K & & \\
 \text{id}_a \hookrightarrow a & \xrightarrow{f} & b & \hookrightarrow \text{id}_b & \mathcal{C}
 \end{array}$$

Powerset

$$\begin{array}{ccccc}
 \{s \mid s \subseteq a\} & \xrightarrow{s \mapsto \{fx \mid x \in s\}} & \{s \mid s \subseteq b\} & \text{Set} \\
 \uparrow P & & \uparrow P & \\
 a & \xrightarrow{f} & b & \text{Set}
 \end{array}$$

Exercise on the functor Powerset

Let $f = \{(1, p), (2, p)\}$ and

$$\begin{array}{ccc} P\{1, 2\} & \xrightarrow{s \mapsto \{fx | x \in s\}} & P\{p, q\} \\ \uparrow P & & \uparrow P \\ \{1, 2\} & \xrightarrow{f} & \{p, q\} \end{array}$$

be given.

- Write step by step how to compute $Pf(\{2\})$ and $Pf(\{1, 2\})$
- Add $Pf(\{2\})$ and $Pf(\{1, 2\})$ to the diagram as elements that belong to $P\{1, 2\}$, and $P\{p, q\}$

Kleene Star Functor

$$\begin{array}{ccc}
 \Sigma^* & \xrightarrow{f^*} & \Gamma^* \\
 \uparrow (-)^* & & (-)^* \uparrow \\
 \Sigma & \xrightarrow{f} & \Gamma
 \end{array}$$

- $\Sigma^* = \{\bullet_\Sigma, \cdot, \text{hom}(\bullet_\Sigma, \bullet_\Sigma)\}$ free monoid on the alphabet Σ :

$$\xrightarrow{\varepsilon \cdot} \in \text{hom}(\bullet_\Sigma, \bullet_\Sigma)$$

$$\xrightarrow{w \cdot} \in \text{hom}(\bullet_\Sigma, \bullet_\Sigma) \quad \text{if } w \in \Sigma$$

$$\xrightarrow{(\sigma \cdot \sigma') \cdot} \in \text{hom}(\bullet_\Sigma, \bullet_\Sigma) \quad \text{if } \xrightarrow{\sigma \cdot}, \xrightarrow{\sigma' \cdot} \in \text{hom}(\bullet_\Sigma, \bullet_\Sigma)$$

$$f^*(w \cdot) = f(w) \quad \text{if } w \in \Sigma$$

$$f^*((\sigma \cdot) \cdot (\sigma' \cdot)) = f^*(\sigma \cdot) \cdot f^*(\sigma' \cdot) \quad \text{if } \sigma \cdot, \sigma' \cdot \in \Sigma^*$$

- Γ^* free monoid analog to Σ^* , but on alphabet Γ

Exercise on Kleene Star Functor

Let

- the following alphabets $\Sigma = \{a, b\}$, $\Gamma = \{c, dd\}$
- the following functions $f(a) = c$, $f(b) = dd$

be given

Problem.

- Can $ccddcc$ be an element of Γ^* generated by the Kleene Star functor $(-)^*$? Justify the answer

Functors in Haskell

Functor in Haskell

Functor $\Leftrightarrow$ map “types to types” + higher-order arrow lifting arrows

```
ghci> :i Functor
type Functor :: (* -> *) -> Constraint -- argument of kind * -> *:
                                         -- map from type to type

class Functor f where
  fmap :: (a -> b) -> f a -> f b      -- higher-order arrow to lift arrows
  (<$) :: a -> f b -> f a             {- "inject" an element of a into fa,
                                         no matter what the element f b is -}
                                         -- at least 'fmap' must be defined
  {-# MINIMAL fmap #-}
```

Source file of reference

Support.CT0300FunctorExamples.hs

Exercises

Diagonal is a functor (Haskell)

Consider the following Haskell code:

```
-- Type constructor of kind * -> *  
newtype Diagonal a = Diagonal {runDiagonal :: (,) a a}
```

Prove that `Diagonal` is a functor by:

- ① defining `fmap`
- ② proving the identity law
- ③ proving the composition law

Sink data-type

Consider the following Haskell code:

```
newtype Sink s a = Sink { runSink :: a -> s }
```

```
instance Functor (Sink s) where  
  fmap :: (a -> b) -> (Sink s a) -> (Sink s b)  
  fmap f g = ...
```

① Is there `fmap :: (a -> b) -> Sink a -> Sink b`?

Is **T** a functor?

```
-- Type constructor of kind * -> *
data T a where
    TL :: a -> T a
    TN :: [T a] -> T
-- unit leaves
10, 11, 12 :: T ()
10 = TL ()
11 = TL ()
12 = TL ()
-- some branches
t1, tr :: [T ()]
t1 = [10, 11]
tr = [12]
-- nest lists to grow branches
t0, t1, t2 :: [[T ()]]
t0 = [t1, tr]
t1 = [[10], tr]
t2 = [[10], [10, 11]]
```

- ① Define **fmap** for “**T a**”
- ② Prove “**fmap** g . fmap f == fmap (g . f)” (composition law)

Support.CT0300FunctorFirstListTree_SOLUTION.hs is restricted

Is `Tree` a functor?

```
-- Type constructor of kind * -> * -> *
data Tree k a where
    TLeaf :: (k -> a) -> Tree k a    -- leaves contain functions
    TNode :: [Tree k a] -> Tree k a -- new tree from a list of trees

-- unit leaves
10, 11, 12 :: Tree k ()
10 = TLeaf (const ())
11 = TLeaf (const ())
12 = TLeaf (const ())

-- some branches
t1, tr :: [Tree k ()]
t1 = [10, 11]
tr = [12]

-- nest lists to grow branches
t0, t1, t2 :: [[Tree k ()]]
t0 = [t1, tr]
t1 = [[10], tr]
t2 = [[10], [10, 11]]
```

① Can we conclude that `Tree k` is a functor?

Support.CT0300FunctorSecondListTree_SOLUTION.hs is restricted



Milewski, B. and I. Tabachnik (2019). *Category Theory for Programmers*.

<https://bartoszmilewski.com/2014/10/28/category-theory-for-programmers-the-preface/>. Lightning Source UK. ISBN: 9780464243878. URL:

<https://books.google.it/books?id=hA55yQEACAAJ>.